

# Sistema de Gestión Bancaria

Metodología Orientada a Objetos

**Siempre estaremos contigo. M.L.A.P.**

**M** Moderno  
**L** Lealtad/Liquidez  
**A** Accesibilidad  
**P** Precisión



ÍNDICE:

1. **Introducción/Objetivo del programa:** Se da una breve descripción de lo que hace el sistema.
2. **Diagrama de Clases:** El diagrama de árbol que define como están construidas las clases
3. **Descripción de las Clases:** Se explica brevemente como los archivos .py su propósito y atributos/métodos clave.
4. **Uso del Sistema (Programa Principal):** Cómo ejecutar e interactuar con tu SistemaBancario.py.
5. **Instrucciones de Ejecución:** Pasos para poner en marcha el programa.

## 1. Introducción y Objetivo del Programa:

Este documento describe el diseño e implementación de un sistema de gestión bancario básico. El objetivo principal de este ejercicio es simular las operaciones fundamentales de un Banco, en el cual se permite la administración de diferentes tipos de cuentas como lo son Débito, Crédito, Nómina, además, de una gestión de la información de ejecutivos bancarios. El sistema con el que está diseñado es para ser modular y extensible, en el cual se utilizan principios de la Programación Orientado a Objetos (POO).

Las funcionalidades clave de este proyecto incluyen:

- Creación, actualización y eliminación de cuentas bancarias.
- Gestión de información de ejecutivos.
- Realización de operaciones bancarias básicas como depósitos, retiros y pagos de crédito/nómina.
- Consultas detalladas sobre los clientes, cuentas y ejecutivos utilizando varios criterios.
- Persistencia de datos mediante archivos CSV.

## 2. Diagrama de clases:

Se presentará con un diagrama de árbol de las clases, el cual sirvió como base para el diseño, creación y la implementación de la estructura del sistema para esta Institución Bancaria. En este diagrama se muestra e ilustra las clases principales, sus atributos, métodos y las relaciones (herencia, composición, etc.) entre ellas.

Para la creación de este diagrama, se extrajeron los atributos de cada Cuenta del Documento dado “Proyecto Final” y se organizaron en una tabla para así poder identificar los atributos de cada cuenta bancaria.

En donde se observaron las similitudes que tenían cada una de ellas (así creando una superclase, la cual se heredaría a sus clases hijas) y cuáles eran sus características únicas (subclases). Finalmente, al realizar este análisis se crea el diagrama de árbol con el cual nos apoyamos para crear las clases.

(Se pueden observar en el documento (PDF) llamado “Proyecto-Programación”, que se añade en el apartado)

### 3.Descripción de las Clases

El Sistema utilizado está compuesto por las siguientes clases, organizadas en diferentes archivos Python para un mejor modularidad:

#### 3.1 Cuenta.py (Superclase Abstracta)

- **Propósito:** Define la estructura base y el comportamiento común para todos los tipos de cuentas bancarias. Es una clase abstracta (ABC) que se asegura que las subclases implementen el método retirar.
- **Atributos Clave (privados):** \_\_n\_cliente, \_\_num\_cliente, \_\_fecha\_apertura, \_\_num\_sucursal, \_\_estado, \_\_correo, \_\_telefono.
- **Métodos Clave:**
  - \_\_init\_\_: Constructor para inicializar los atributos básicos de una cuenta.
  - @property (getters y setters): Métodos para acceder y modificar los atributos de forma controlada.
  - retirar (abstracto): Método a ser implementado por las subclases para manejar las particularidades del retiro según el tipo de cuenta.
  - \_\_str\_\_, \_\_iter\_\_, \_\_hash\_\_, \_\_eq\_\_: Métodos para representación en cadena, iteración, hashing y comparación de objetos.

#### 3.2 CuentaDN.py (Subclase de Cuenta – Base para Débito/Nómina)

- **Propósito:** Hereda de Cuenta y añade atributos y comportamientos comunes a las cuentas de débito y nómina, como el número de cuenta y el saldo.
- **Atributos Clave (privados):** \_\_num\_cuenta, \_\_saldo.
- **Métodos Clave:**
  - \_\_init\_\_: Constructor que llama al constructor de la superclase y añade los atributos específicos.
  - @property (getters y setters): Para num\_cuenta y saldo.
  - depositar: Permite añadir un monto al saldo de la cuenta.
  - retirar: Implementa el método abstracto de Cuenta para reducir el saldo (si hay fondos suficientes).
  - \_\_str\_\_, \_\_iter\_\_, \_\_hash\_\_, \_\_eq\_\_: Métodos mágicos para la clase.

#### 3.3 CuentaD (Subclase de CuentaDN –Cuenta de Débito)

- **Propósito:** Representa una cuenta de débito, añadiendo la funcionalidad de una fecha de corte específica.
- **Atributos Clave (privados):** \_\_fecha\_corte.

- **Métodos Clave:**

- `__init__`: Constructor que llama al constructor de CuentaDN y añade fecha\_corte.
- `@property` (getter y setter): Para fecha\_corte.
- Hereda depositar y retirar de CuentaDN.
- `__str__`, `__iter__`, `__hash__`, `__eq__`: Métodos mágicos para la clase.

### 3.4 CuentaN.py (Subclase de CuentaDN – Cuenta de Nómina)

- **Propósito:** Representa una cuenta de nómina, con funcionalidades específicas para la recepción de pagos de nómina y restricciones en depósitos directos.
- **Atributos Clave (privados):** `__rfc_empresa`, `__n_empresa`, `__fecha_dep`.
- **Métodos Clave:**
  - `__init__`: Constructor que llama al constructor de CuentaDN y añade los atributos específicos de nómina.
  - `@property` (getters y setters): Para `rfc_empresa`, `n_empresa` y `fecha_dep`.
  - `depositar` (sobrescrito): Restringe los depósitos directos.
  - `recibir_pago_nomina`: Método específico para que la empresa autorizada deposite la nómina.
  - Hereda retirar de CuentaDN.
  - `__str__`, `__iter__`, `__hash__`, `__eq__`: Métodos mágicos para la clase.

### 3.5 CuentaC (Subclase de Cuenta – Cuenta de Crédito)

- **Propósito:** Representa una cuenta de crédito, con funcionalidades para gestionar el importe y el monto de crédito utilizado, así como fechas de pago y vencimiento.
- **Atributos Clave (privados):** `__num_tarjeta`, `__imp_credito`, `__mon_credito`, `__fecha_pago`, `__fecha_ven`.
- **Métodos Clave:**
  - `__init__`: Constructor que llama al constructor de Cuenta y añade los atributos específicos de crédito.
  - `@property` (getters y setters): Para todos los atributos de crédito.
  - `retirar`: Implementa el método abstracto de Cuenta para realizar avances de efectivo (con comisión).
  - `realizar_pago_tarjeta`: Permite al cliente realizar pagos a la tarjeta, incluyendo penalizaciones si el pago no es completo.
  - `__str__`, `__iter__`, `__hash__`, `__eq__`: Métodos mágicos para la clase.

### 3.6 Ejecutivo.py

- **Propósito:** Define la estructura y el comportamiento de un objeto Ejecutivo, que representa a un empleado del banco.
- **Atributos Clave (privados):** \_\_num\_empleado, \_\_rfc, \_\_n\_empleado, \_\_fecha\_nac, \_\_direccion, \_\_telefono, \_\_sueldo\_m.
- **Métodos Clave:**
  - \_\_init\_\_: Constructor para inicializar los atributos del ejecutivo.
  - @property (getters y setters): Métodos para acceder y modificar los atributos.
  - \_\_str\_\_, \_\_iter\_\_, \_\_hash\_\_, \_\_eq\_\_: Métodos mágicos para representación, iteración, hashing y comparación.

### 3.7 SistemaBancario.py (Clase principal para la gestión de la Institución Bancaria)

- **Propósito:** Contiene la lógica central del sistema. Actúa como el controlador principal que interactúa con las clases de cuentas y ejecutivos. Maneja la carga y guardado de datos desde/hacia archivos CSV, las consultas, y las operaciones de alta, actualización, eliminación y transacciones.
- **Atributos Clave (privados):** \_\_cuentas (lista de objetos Cuenta), \_\_ejecutivos (lista de objetos Ejecutivo), \_\_cuentas\_borradas, \_\_ejecutivos\_borrados.
- **Métodos Clave:**
  - \_\_init\_\_: Inicializa las listas de cuentas y ejecutivos.
  - cargar\_cuentas(): Lee y carga los datos de cuentas desde cuentas.csv.
  - cargar\_ejecutivos(): Lee y carga los datos de ejecutivos desde ejecutivos.csv.
  - consultar\_cliente(), consultar\_cuenta(), consultar\_ejecutivo(): Métodos para realizar búsquedas complejas.
  - alta\_actualizar\_cuenta(), alta\_actualizar\_ejecutivo(): Permite crear nuevas entradas o modificar existentes.
  - registrar(): Guarda los datos actuales de cuentas en cuentas.csv.
  - eliminar\_cuentas(), eliminar\_ejecutivos(): Permite eliminar entradas y guardarlas en archivos \_borradas.csv.
  - buscar\_cuenta\_operacion(): Método auxiliar para encontrar una cuenta por su identificador.
  - realizar\_deposito\_deb(): Gestiona depósitos en cuentas de débito.

- realizar\_pago\_nomina(): Gestiona depósitos de nómina (solo por empresas autorizadas).
- realizar\_retiro(): Gestiona retiros de cualquier tipo de cuenta (polimórfico).
- realizar\_pago\_credito(): Gestiona pagos a tarjetas de crédito.

## 4. Uso del Sistema (Programa Principal)

El programa principal, que se ejecutaría desde el archivo PruebaSistema.py, es el punto de entrada para el usuario. Su función es instanciar la clase InstitucionBancaria y presentar un menú interactivo que permite al usuario seleccionar su rol (Ejecutivo o Cliente) para acceder a las funcionalidades específicas proporcionadas por SistemaBancario.py.

El Menú se puede encontrar en el archivo PruebaSistema.py. El cual carga automáticamente los archivos.csv, para posteriormente desplegarse un menu con las opciones en cual elijes si eres un ejecutivo o un cliente, en donde usted como persona elije que clase de persona es se explicara mas adelante un poco acerca de esto.

## 5. Instrucciones de Ejecución

Para ejecutar el sistema, se necesita tener todos los archivos .py (Cuenta.py, CuentaC.py, CuentaDN.py, CuentaD.py, CuentaN.py, Ejecutivo.py, SistemaBancario.py, y PruebaSistema.py) en el mismo directorio.

### 1. Requisitos:

- Tener Phyton.

### 2. Archivos de Datos (Carga Automática):

- El sistema gestiona automáticamente los archivos de datos cuentas.csv y ejecutivos.csv.
- Al iniciar el programa, intentará cargar los datos existentes de estos archivos. Si los archivos no se encuentran en el directorio, el sistema los creará automáticamente según sea necesario cuando se realicen operaciones que persistan datos (ej. dar de alta una cuenta, registrar cuentas, etc.). Por lo tanto, **no es necesario crearlos manualmente antes de la ejecución.**

### 3. Ejecutar el programa:

Para ejecutar el programa necesitamos abrir el archivo PruebaSistema.py y correr el archivo

- Al ejecutar el programa, se mostrará un menú principal en la terminal, pidiéndote que indiques si eres un cliente o un ejecutivo.

- Selecciona la opción deseada (1 para Ejecutivo, 2 para Cliente, 3 para Salir) y presiona Enter.

- Dependiendo de tu elección, se te presentará un submenú con las operaciones específicas para tu rol.

Como cliente, se despliegan las opciones:

- Depositar dinero
- Retirar dinero
- Pago de Tarjeta
- Regresar

Los cuales no llevan a sus respectivos metodos para realizar las operaciones.

Como ejecutivo, se abren las opciones:

- Consultar clientes
- Consultar cuentas.
- Consultar ejecutivos
- Dar de alta una nueva cuneta o actual
- Registrar el alta de una nueva cuenta
- Dar de alta un (s) nuevo (a) ejecutivo o actualizar una existente
- Eliminar cuenta
- Eliminar ejecutivo
- Regresar

Las cuales al utilizar cualquier opcion nos lleva a las acciones que deseamos hacer y seguimos los pasos que nos solicitan.

- Sigue las instrucciones en pantalla para realizar consultas, altas, actualizaciones, eliminaciones o transacciones bancarias, etc.

